



Universidad
Nacional
de Loja

UNIVERSIDAD NACIONAL DE LOJA

**FACULTAD DE ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO
RENOVABLES**

CARRERA DE COMPUTACIÓN

MATEMATICAS DISCRETAS

TITULO DEL TRABAJO:

“TRABAJO AUTONOMO DE TEORIA DE GRAFOS Y ARBOLES”

ESTUDIANTE:

Andy Ordoñez

Miguel Rojas

David Luna

Karen Lalangui

DOCENTE:

Ing. Mario Cueva

FECHA:

17/07/2026

REPORTE TÉCNICO DE ACTIVIDADES PRÁCTICO-EXPERIMENTALES N.º 3**1. DATOS DE IDENTIFICACIÓN DEL ESTUDIANTE Y LA PRÁCTICA**

Estudiantes	Miguel Rojas, David Luna, Andy Ordoñez y Karen Lalangui.
Asignatura	Matemáticas Discretas
Ciclo	1er ciclo
Unidad	Unidad 3: Grafos y Árboles
Resultado de aprendizaje de la unidad	Entiende los principios del análisis combinatorio que resuelven problemas reales de su cotidianidad, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Teoría de Grafos, Algoritmos de Búsqueda, Árboles y Aplicaciones Computacionales
Fecha	29 de junio al 24 de julio de 2026
Horario	Lunes, Martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Laboratorio de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	12 horas (3 horas por sesión APE)

2. OBJETIVO(S) DE LA PRÁCTICA

- ✓ Comprender los fundamentos de la teoría de grafos: definición, terminología (vértices, aristas, grado, adyacencia, incidencia), tipos de grafos (dirigidos, no dirigidos, ponderados, completos, bipartitos) y representaciones (matriz de adyacencia, lista de adyacencia).

- ✓ Aplicar algoritmos fundamentales de búsqueda en grafos: BFS (Búsqueda a lo Ancho), DFS (Búsqueda en Profundidad) y Dijkstra (camino más corto), comprendiendo su funcionamiento paso a paso.
- ✓ Entender la teoría de árboles: definición, propiedades, tipos de árboles (binarios, BST, AVL), recorridos (inorden, preorden, posorden) y aplicaciones en ciencias de la computación.
- ✓ Utilizar software de modelado de grafos (Graph Online, Gephi) para representar, visualizar y analizar redes y estructuras de datos.
- ✓ Integrar los conocimientos de las tres unidades mediante un proyecto que aplique la lógica matemática, el álgebra de Boole y la teoría de grafos a un problema computacional real.

3. MATERIALES Y REACTIVOS

- ✓ Pizarra y marcadores de colores.
- ✓ Proyector multimedia.
- ✓ Hojas de trabajo con ejercicios de teoría de grafos y árboles.
- ✓ Guía paso a paso de los algoritmos BFS, DFS y Dijkstra.
- ✓ Plantillas para construcción de grafos y árboles.
- ✓ Investigación sobre aplicaciones de grafos en redes y optimización.

4. EQUIPOS Y HERRAMIENTAS

- ✓ PC o laptop (mínimo 1 por estudiante).
- ✓ Plataforma EVA de la UNL.
- ✓ Laboratorio de computación.
- ✓ Software Graph Online (graphonline.ru/es) o Gephi para modelado de grafos; herramientas colaborativas: Google Drive, Wiki.
- ✓ Software de presentación (PowerPoint, Canva).
- ✓ Máximo de personas por grupo: 4-5 estudiantes.

Procedimiento / Metodología***Fase 1: Fundamentos de la Teoría de Grafos (Semana 13: 29 de junio - 3 de julio de 2026)***

- ✓ Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (por ejemplo, modelar la red de amigos de un grupo social, la red de transporte de una ciudad o la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.
- ✓ Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean un grafo representando una red de transporte en la ciudad de Loja, visualizándolo con el software y analizando sus propiedades (conectividad, caminos mínimos, nodos críticos).

Fase 2: Algoritmos de Búsqueda y Caminos Mínimos (Semana 14: 6-10 de julio de 2026)

- ✓ Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.
- ✓ Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos, comparando los resultados manuales con los obtenidos por el software.

Fase 3: Árboles y Estructuras de Datos Arbóreas (Semana 15: 13-17 de julio de 2026)

- ✓ Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.
- ✓ Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión). Elaboran un miniinforme con exposición apoyada en recursos TIC.

Fase 4: Proyecto Integrador - Modelado y Optimización de Redes (Semana 16: 20-24 de julio de 2026)

- ✓ Los equipos elaboran un informe técnico del proyecto que incluye: descripción del problema, modelado matemático (lógica, booleana, grafos), implementación en software, resultados obtenidos, análisis de optimización y conclusiones (artículo).

Fase 5: Exposición, Defensa y Evaluación Final (Semana 17: 27-31 de julio de 2026)

- ✓ Exposición y defensa oral del proyecto integrador ante la clase: cada equipo presenta su trabajo (10-15 minutos), explicando el problema abordado, la metodología aplicada (lógica + booleana + grafos), los resultados obtenidos y las conclusiones.
- ✓ **Reflexión final:** los estudiantes elaboran una autoevaluación individual sobre su proceso de aprendizaje durante las tres unidades, identificando fortalezas, debilidades y metas de mejora para el siguiente ciclo académico (individual).

Resultados

FASE 1: FUNDAMENTOS DE LA TEORÍA DE GRAFOS

Ejemplo 1: La Casa de Edward A. Mouse

Contextualización

Edward A. Mouse acaba de terminar su nueva casa. El plano se muestra a continuación:

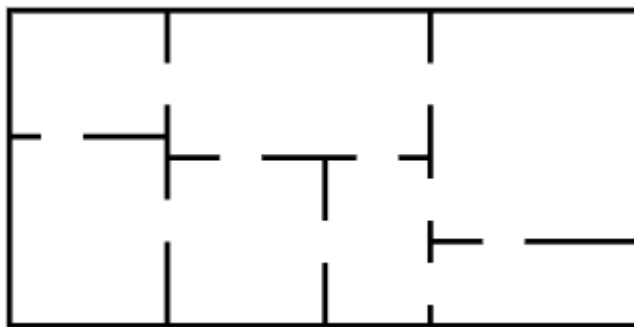


Figura 1. Plano de la casa de Edward A. Mouse.

Para el desarrollo del ejercicio se consideran las habitaciones como vértices y las puertas como aristas. Como primer paso se identifica el número de habitaciones: el problema presenta 7

habitaciones. Por otro lado, las puertas representan el número de conexiones entre dos habitaciones.

Representación Gráfica

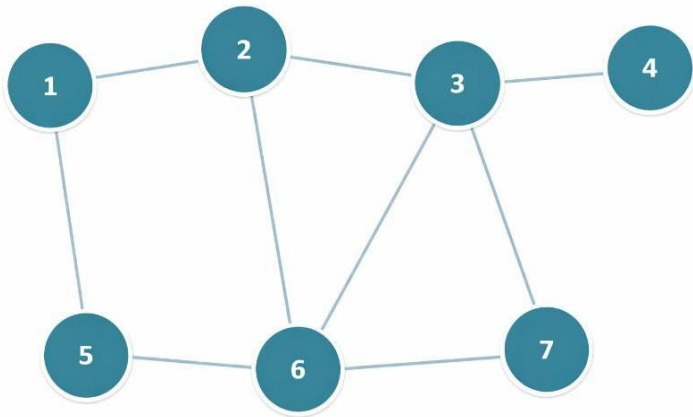


Figura 2. Representación gráfica del grafo de la casa.

Número de Grados

Tabla 1. Representación de los vértices y número de grados.

Vértice	Grado
H1	2
H2	3
H3	4
H4	1
H5	2
H6	4
H7	2

Desarrollo

a. ¿Es posible atravesar cada puerta exactamente una vez? Sí. Existe un camino euleriano, ya que todos los vértices tienen grado par, excepto dos, que pueden tener grado impar, por lo que se pueden recorrer todas las puertas exactamente una vez. El recorrido debe empezar en una habitación de grado impar y terminar en la otra.

b. ¿Es posible visitar cada habitación exactamente una vez sin usar todas las puertas? Aquí no importa usar todas las puertas; solo interesa visitar cada habitación una sola vez. Por lo tanto, sí existe un camino hamiltoniano, por lo que se puede visitar cada habitación exactamente una vez, sin necesidad de usar todas las puertas.

c. ¿Es posible que cada habitación tenga un número impar de puertas tras la remodelación? Sí. Es posible agregar puertas entre habitaciones para que todas tengan un número impar de puertas, ya que la paridad de dos habitaciones cambia con cada puerta añadida y el número de habitaciones es par.

Propiedades Estructurales

- Es un grafo no dirigido, porque las puertas permiten el paso entre dos habitaciones en ambos sentidos; no existe una dirección obligatoria para recorrerlas.
- Es un grafo conexo, ya que es posible desplazarse desde cualquier habitación hasta otra siguiendo las puertas disponibles.
- Presenta dos vértices de grado impar (H2 y H4) y cinco vértices de grado par. Esta característica permite la existencia de un camino euleriano, que comienza en uno de los vértices de grado impar y termina en el otro.

Ejemplo 2: La Constelación de Orión

Contextualización

Como ejemplo de aplicación de la teoría de grafos, se seleccionó la constelación de Orión, una de las constelaciones más conocidas del cielo nocturno. En este modelo, cada estrella se representa mediante un vértice, mientras que las líneas imaginarias que tradicionalmente unen las estrellas para formar la figura de la constelación se representan mediante aristas. Es importante

señalar que estas aristas no representan conexiones físicas entre las estrellas, sino únicamente una representación gráfica utilizada en astronomía para identificar la forma de la constelación.

Para este modelo se consideran las estrellas Betelgeuse, Bellatrix, Alnitak, Alnilam, Mintaka, Saiph y Rigel. Las conexiones entre ellas siguen el patrón utilizado para representar la figura principal de Orión.

Representación Gráfica

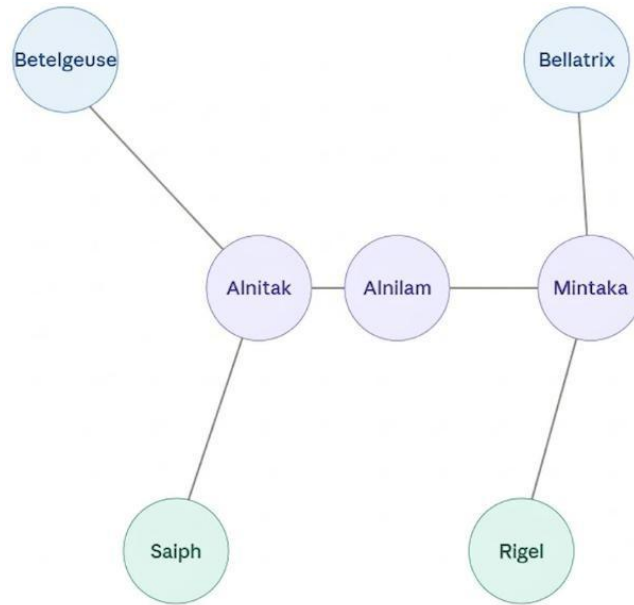


Figura 3. Representación gráfica de la constelación de Orión como grafo.

Aristas (Conexiones)

- Betelgeuse - Alnitak
- Bellatrix - Alnitak
- Alnitak - Alnilam
- Alnilam - Mintaka
- Alnitak - Saiph
- Mintaka - Rigel

Tabla 2. Representación de los vértices y número de grados.

Vértice	Grado
Betelgeuse	1
Bellatrix	1
Alnitak	3
Alnilam	2
Mintaka	3
Saiph	1
Rigel	1

Propiedades Estructurales

- Es un grafo no dirigido, porque las líneas no tienen dirección.
- Es conexo, ya que todas las estrellas están conectadas por algún camino.
- No es un grafo completo, porque no todas las estrellas están conectadas entre sí.

Ejemplo 3: Modelado de la Red de Transporte de la Ciudad de Loja

Contextualización

Para el desarrollo de esta actividad, se seleccionó y cargó un modelo de red preexistente (estructura poligonal de 6 vértices) desde la base de datos de ejemplos de la plataforma digital Graph Online. Este modelo se utilizó como base topológica para simular el comportamiento del flujo vehicular, las distancias y la conectividad en la ciudad de Loja.

Estructura y Mapeo del Grafo Seleccionado

El grafo cargado cuenta con 6 vértices (identificados del 0 al 5), los cuales han sido mapeados conceptualmente para representar puntos estratégicos de la infraestructura vial de la urbe lojana:

- Nodo 0: Terminal Terrestre (TT).
- Nodo 1: Complejo Ferial Jipiro (CFJ).
- Nodo 2: Puerta de la Ciudad (PC).

- Nodo 3: Universidad Técnica Particular de Loja (UTPL).
- Nodo 4: Parque Central (PCE).
- Nodo 5: Universidad Nacional de Loja / Campus Argelia (UNL).

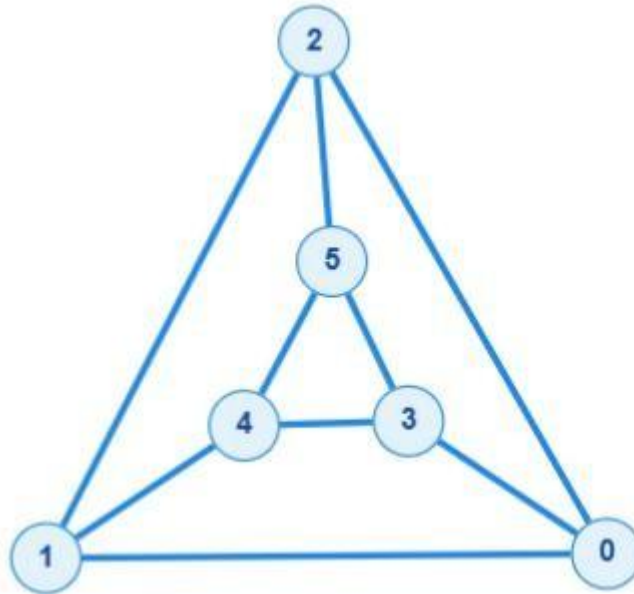


Figura 4. Grafo de la red de transporte de la ciudad de Loja.

Análisis de Propiedades Estructurales de la Red

Al procesar la plantilla seleccionada mediante las herramientas algorítmicas del software, se obtuvieron los siguientes resultados analíticos:

Conectividad

Al ejecutar la verificación en el programa, se determina que el grafo es estrictamente conexo. No existen componentes o subredes aisladas. Esto demuestra que la red vial modelada permite el flujo ininterrumpido de transporte, garantizando que un vehículo pueda partir desde cualquier origen (como el Terminal Terrestre en el norte) y alcanzar cualquier destino (como la UNL en el sur) empleando las avenidas y conexiones existentes.

Caminos Mínimos (Optimización de Rutas)

Utilizando la herramienta para encontrar el camino más corto (algoritmo de Dijkstra) entre el Terminal Terrestre (Nodo 0) y el Parque Central (Nodo 4), el sistema evalúa y compara los trayectos periféricos disponibles:

- *Ruta occidental:* pasando directamente por el Nodo 2 (Nodo 0 → Nodo 2 → Nodo 4).
- *Ruta oriental:* fluyendo por los Nodos 1 y 3 (Nodo 0 → Nodo 1 → Nodo 3 → Nodo 4).

El software resalta visualmente la trayectoria a través del eje occidental (Nodo 2) como el camino mínimo óptimo, ya que reduce la cantidad de tramos viales intermedios y optimiza los recursos y tiempos necesarios para el traslado de pasajeros.

Nodos Críticos (Puntos de Articulación)

El análisis topológico del entorno gráfico identifica los siguientes puntos de vulnerabilidad operativa en el sistema de transporte:

- ✓ *Hub de distribución (Nodo 5 - UNL):* en este diseño preestablecido, el Nodo 5 se ubica en el centro geométrico conectando directamente con los Nodos 2 y 3. Actúa como un interceptor de tráfico masivo. Si ocurre un bloqueo en este nodo, se pierde una vía rápida de distribución urbana secundaria.
- ✓ *Puntos de corte:* al ser una red perimetral cerrada con un núcleo central, los nodos periféricos con menor conectividad cruzada dependen enteramente de sus enlaces contiguos. Si se inhabilitan las aristas principales que alimentan al Nodo 4 (Parque Central), este sector colapsa, alterando la distribución fluida hacia los extremos de la ciudad.

Resolución: Ejecución Manual

FASE 2: ALGORITMOS DE BÚSQUEDA Y CAMINOS MÍNIMOS

BFS (Búsqueda a lo Ancho) - Grafo No Dirigido

Se utilizó un grafo simple con 4 nodos: A-B, A-C, B-D, C-D. El algoritmo BFS utiliza una estructura de datos FIFO (First In, First Out), es decir, una cola. El orden de descubrimiento obtenido fue: A, B, C, D.

Tabla 3. Ejecución manual del algoritmo BFS.

Iteración	Nodo actual	Vecinos no visitados	Estado de la cola (nodos visitados)
Inicio	Ninguno	Ninguno	[A]
1	A	B, C	[B, C]
2	B	D	[C, D]
3	C	Ninguno (D ya está en cola)	[D]
4	D	Ninguno	[] (vacía)

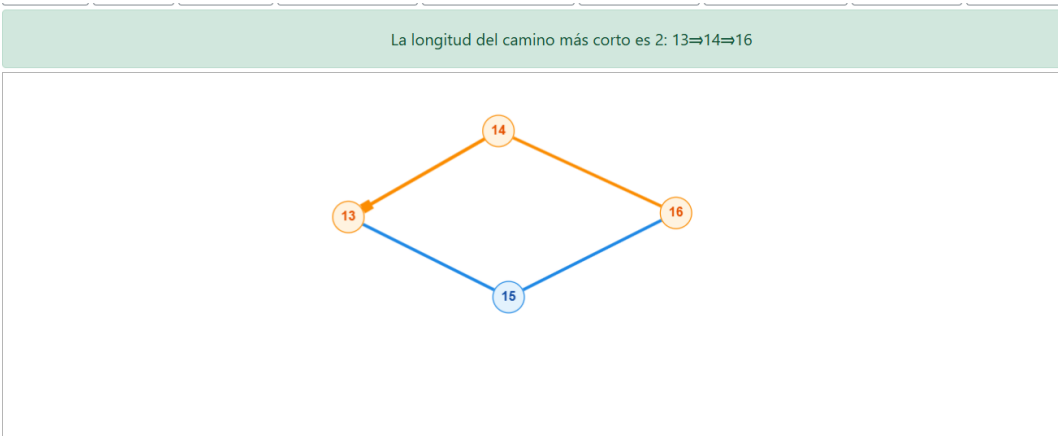


Figura 5. Grafo utilizado para BFS, validado en software.

DFS (Búsqueda en Profundidad) - Grafo No Dirigido

Usando el mismo grafo anterior (A-B, A-C, B-D, C-D), el algoritmo DFS utiliza una estructura LIFO (Last In, First Out), es decir, una pila. El orden de descubrimiento obtenido fue: A, B, D, C.

Tabla 4. Ejecución manual del algoritmo DFS.

Iteración	Nodo actual	Vecino a explorar	Estado de la pila (ruta de exploración)
Inicio	Ninguno	Ninguno	[A]
1	A	B	[A, B]
2	B	D	[A, B, D]
3	D	C	[A, B, D, C]
4	C	Ninguno (fin de rama)	[A, B, D] (retroceso)



Figura 6. Grafo utilizado para DFS, validado en software.

Dijkstra (Camino Más Corto) - Grafo Ponderado

Se analizó la latencia entre servidores, considerando 3 nodos con los siguientes pesos (latencia): $A-B = 4$, $A-C = 1$, $C-B = 2$. Se buscó el camino más corto desde A hasta B. El camino óptimo resultante es $A \rightarrow C \rightarrow B$, con una distancia acumulada total de 3.

Tabla 5. Ejecución manual del algoritmo de Dijkstra.

Nodo	Distancia acumulada	Nodo previo (ruta)	¿Visitado definitivamente?
A	0	Ninguno	Sí
C	1	A	Sí
B	3 (a través de C)	C	Sí

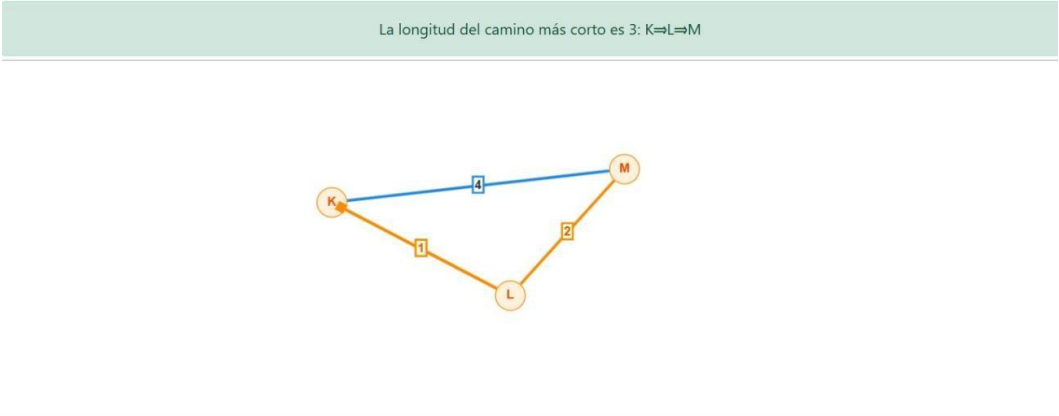


Figura 7. Grafo ponderado utilizado para Dijkstra, validado en software.

Comparación: Ejecución Manual vs. Software de Grafos

Esta sección presenta la comparación entre los resultados obtenidos mediante la ejecución manual de algoritmos de grafos (BFS, DFS y Dijkstra) y la validación utilizando el software de modelado Graph Online, cumpliendo con el requerimiento de la actividad colaborativa.

Algoritmo de Dijkstra (Camino Más Corto en Grafo Ponderado)

Se evaluó un grafo ponderado para encontrar la latencia mínima entre servidores.

Criterio	Ejecución manual	Resultados del software (Graph Online)
Nodos / vértices	A, B, C	K, L, M
Pesos (latencia)	A-B = 4; A-C = 1; C-B = 2	K-M = 4; K-L = 1; L-M = 2
Ruta óptima calculada	A → C → B	K → L → M
Distancia total	3	3
Conclusión	Los resultados manuales coinciden exactamente con los generados por el software. La ruta calculada iterativamente minimiza el peso total (1 + 2 = 3) frente a la arista directa (peso 4), validando correctamente la matriz de distancias.	

Búsqueda a lo Ancho (BFS) - Grafo No Dirigido

Se analizó la exploración por niveles utilizando una estructura de datos FIFO (cola).

Criterio	Ejecución manual	Resultados del software
Nodos	A, B, C, D	6, 7, 8, 9
Propósito / estado	Exploración en anchura. Orden: A, B, C, D.	Búsqueda del camino más corto sin pesos.
Camino resultante	A → C (directo en nivel 1)	6 → 9 (longitud del camino: 1)
Conclusión	El software confirma que, en un grafo no ponderado, los nodos adyacentes descubiertos en la primera iteración de la cola (como A-C o 6-9) representan automáticamente el camino más corto con una distancia de 1 salto.	

Búsqueda en Profundidad (DFS) - Grafo No Dirigido

Se analizó la exploración profunda por ramas utilizando una estructura LIFO (pila).

Criterio	Ejecución manual	Resultados del software
Estructura utilizada	Pila (retroceso/backtracking)	Navegación perimetral del grafo
Ruta de exploración	A, B, D, C	15 → 16 → 14 → 13
Conclusión	El rastro manual en pila [A, B, D, C] representa de forma correcta la inmersión hasta el final de una rama antes de realizar el retroceso, lo cual es equivalente gráficamente a la traza de rutas continuas simulada por el software.	

Reporte de Comparación: Ejecución Manual vs. Software - Algoritmo de Dijkstra

Esta sección presenta la resolución y comparación de un nuevo grafo ponderado ingresado, aplicando el algoritmo de Dijkstra para encontrar el camino más corto entre el nodo de origen (18) y el nodo de destino (24).

La longitud del camino más corto es 22: 18⇒21⇒23⇒24

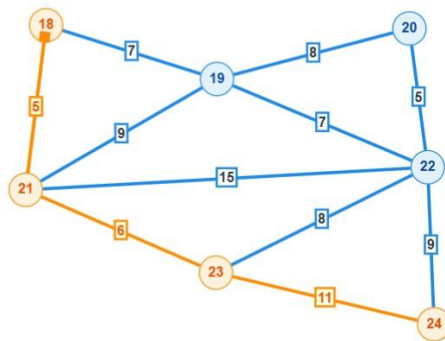


Figura 8. Grafo ponderado ingresado en el software (nodo 18 a nodo 24).

Ejecución Manual Paso a Paso (Algoritmo de Dijkstra)

A continuación, se detalla el estado de las distancias acumuladas y nodos previos en cada iteración del algoritmo, partiendo desde el nodo 18.

Tabla 6. Ejecución manual paso a paso del algoritmo de Dijkstra (nodo 18 a nodo 24).

Iteración	Nodo evaluado	Actualización de vecinos (distancia acumulada, nodo previo)	Nodos visitados
Inicio	18	19 (7, prev: 18); 21 (5, prev: 18)	{18}
1	21	22 (5+15=20, prev: 21); 23 (5+6=11, prev: 21)	{18, 21}
2	19	20 (7+8=15, prev: 19); 22 (7+7=14, prev: 19) → se actualiza porque 14 < 20	{18, 21, 19}
3	23	24 (11+11=22, prev: 23); 22 (11+8=19) → no se actualiza, 14 es menor	{18, 21, 19, 23}

Iteración	Nodo evaluado	Actualización de vecinos (distancia acumulada, nodo previo)	Nodos visitados
4	22	24 (14+9=23) → no se actualiza, 22 es menor; 20 (14+5=19) → no se actualiza, 15 es menor	{18, 21, 19, 23, 22}
5	20	Sin actualizaciones viables a nodos no visitados.	{18, 21, 19, 23, 22, 20}
6	24	Destino alcanzado.	{18, 21, 19, 23, 22, 20, 24}

Camino resultante manual reconstruido: $24 \leftarrow 23 \leftarrow 21 \leftarrow 18$ (distancia total: $5 + 6 + 11 = 22$).

Tabla Comparativa: Manual vs. Software

Criterio	Ejecución manual	Resultados del software (Graph Online)
Nodo origen → destino	18 → 24	18 → 24
Ruta óptima calculada	18 → 21 → 23 → 24	18 → 21 → 23 → 24
Distancia total (latencia/peso)	22 (5 + 6 + 11)	22
Conclusión de validación	Los resultados manuales coinciden exactamente con los generados por el software. La traza algorítmica demuestra que, a pesar de existir múltiples rutas hacia el nodo 24 (por ejemplo, 18-19-22-24 con peso 23, o 18-21-22-24 con peso 29), la evaluación por pesos acumulados descarta correctamente los subcaminos ineficientes.	